Programming Reference: Python Fundamentals & Advanced Concepts

Educational Series

December 29, 2025

1 Introduction to Python

Python is a high-level, interpreted, general-purpose programming language. Its design philosophy emphasizes **code readability** and the use of significant whitespace for block delimitation.

2 Advanced Functional Tools

2.1 List Comprehensions

List comprehensions provide a concise way to create lists based on existing iterables.

```
# Syntax: [expression for item in iterable if condition]
squares = [x**2 for x in range(10) if x % 2 == 0]
# Result: [0, 4, 16, 36, 64]
```

2.2 Lambda Functions

Anonymous, one-line functions defined using the `lambda` keyword.

```
add = lambda x, y: x + y
print(add(5, 3)) # Output: 8
```

3 Decorators

Decorators allow you to modify the behavior of a function or class. They "wrap" another function to extend its behavior without permanently modifying it.

```
def my_decorator(func):
    def wrapper():
        print("Something before the function.")
        func()
        print("Something after the function.")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")
```

4 Generators and Iterators

Generators are functions that return an iterable set of items, one at a time, using the `yield` keyword. They are **memory-efficient** because they do not store the entire list in memory.

```
def count_up_to(n):
    count = 1
    while count <= n:
        yield count
        count += 1

counter = count_up_to(5)
# Items are generated only when requested
```

5 Context Managers (The 'with' Statement)

Context managers ensure that resources are properly acquired and released (e.g., closing a file or a database connection).

```
# Ensures the file is closed even if an error occurs
with open("test.txt", "w") as f:
    f.write("Hello World")
```

6 Object-Oriented Programming (OOP) - Advanced

- **Dunder Methods:** Special methods like `__init__`, `__str__`, and `__repr__` that allow you to emulate built-in behavior.
- **Inheritance:** Python supports multiple inheritance.
- **Properties:** Using `@property` to create managed attributes (getters/setters).

7 Error and Exception Handling

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error: {e}")
else:
    print("Success!")
```

8 Standard Library Highlights

- `itertools`: Functions creating iterators for efficient looping.
- `collections`: High-performance container data types (e.g., `namedtuple`, `Counter`).
- `threading/multiprocessing`: For concurrent and parallel execution.